# CUDA Profiling Lab Report

Tim Li

October 22, 2026

## 1 Introduction

This lab examined CUDA performance at the hardware, application, and kernel levels on an NVIDIA Jetson Orin Nano development platform. Following the lab instructions [1], we first checked the power mode and fixed the clocks to reduce variation from frequency scaling. A device-query program then gave the hardware resource limits. Vector addition served as a simple workload for comparing three views of the same run: compiler resource reports, CUDA event timing, and the kernel and memory-copy durations recorded by Nsight Systems. Finally, a memory-stride experiment used Nsight Compute counters to relate execution time to the addresses accessed by neighboring threads. We also used `tegrastats` to observe GPU activity, temperature, and device input power while the programs ran.

The goal was to explain measured performance rather than assume that a larger block, higher occupancy, or higher hardware utilization makes a program faster. We made two main comparisons: the cost of data movement relative to the vector-addition kernel, and the effect of changing the read stride while keeping the same kernel and launch configuration.

## 2 Procedure, Observations, and Discussion

### 2.1 Source Code

The three source files are available at a fixed repository revision [2]. The links identify the submitted version, so the report does not need to repeat full listings.

- `device_query.cu`: hardware and available-memory query.
- `vector_add.cu`: vector addition, event timing, and a result spot check.
- `memory_stride.cu`: comparison of read strides 1 and 32.

Revision: `a4d7bb367b202c1fa3f8f605ba8168c53a924af0`. The analysis below uses the captured experimental outputs, and the repository links serve as the source-code record.

### 2.2 Experimental Setup and Hardware Resources

The experiment used the existing `~/ESE366/profiling` directory. We queried the power mode, fixed the clocks, and checked the resulting settings with:

```
sudo nvpmodel -q
sudo jetson_clocks
sudo jetson_clocks --show
```

The system identified itself as an NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super. It ran in the 25 W power mode with six CPU cores online at 1.344 GHz, and the GPU minimum, maximum, and current frequencies were all 918 MHz. The EMC frequency was 3.199 GHz with frequency override enabled. The power-mode label names the selected operating mode, not a measured power draw.

Table 1 gives the hardware budget from the device query.

Table 1: Device-query results. Available memory is a snapshot of system state.

| Property | Measured value |
|---|---|
| GPU name | Orin |
| Compute capability | 8.7 |
| Streaming multiprocessors (SMs) | 8 |
| Warp size | 32 threads |
| Maximum threads per block | 1024 |
| Maximum resident threads per SM | 1536 |
| Registers per SM | 65,536 |
| Shared memory per block | 49,152 bytes |
| Free memory | 3713.39 MiB |
| Total memory | 7607.37 MiB |

Having eight SMs does not limit a grid to eight blocks. Each block runs on one SM, and an SM can hold several blocks when its resources allow. For 256-thread blocks, the 1536-thread limit alone allows six blocks per SM, or 48 warps. Registers, shared memory, and other limits must also be checked. Resident warps are also only candidates for scheduling, because they do not all issue instructions at the same time.

The Jetson uses integrated system memory, but the explicit host and device allocations and `cudaMemcpy` calls in our program still cost measurable time. The host-to-device and device-to-host labels therefore do not indicate transfers over a discrete GPU's PCIe link.

### 2.3 Vector Addition: Timing and Compiler Resources

The vector-addition program used $N = 2^{20} = 1{,}048{,}576$ elements, initialized with $A[i] = i$ and $B[i] = 2i$. Each thread computed

$$C[i] = A[i] + B[i], \qquad i = \texttt{blockIdx.x}\,\texttt{blockDim.x} + \texttt{threadIdx.x}. \tag{1}$$

A bounds check protected indices beyond $N$. With 256 threads per block, the launch had 4096 blocks.

```
nvcc -arch=sm_87 --ptxas-options=-v vector_add.cu -o vector_add 2>&1 | tee
    vector_add_compile.txt
./vector_add | tee vector_add_output.txt
```

The compiler reported 12 registers per thread, a zero-byte stack frame, no spill stores or loads, and 380 bytes of constant memory in bank 0. These describe the compiled kernel, not its numerical output. The explicit `sm_87` target matches the queried compute capability.

The standalone run printed an event interval of **0.686816 ms** and `c[100] = 300.000000`, which agrees with $100 + 200 = 300$. This spot check does not verify every element. The input copies came before the start event and the output copy came after the stop event. The run had no warm-up and gave a single measurement, not a mean over repeated runs.

## 2.4 Application-Level Profiling with Nsight Systems

We profiled the application with the following command [4]:

```
nsys profile --stats=true -o vector_add_profile ./vector_add 2>&1 | tee vector_add_nsys.
    txt
```

Table 2 summarizes the recorded GPU operations, with the original nanosecond values converted to milliseconds. The two input copies moved a total of 8 MiB, and the output copy moved 4 MiB. The copies took

Table 2: GPU operation durations from the Nsight Systems run.

| Operation | Count | Total duration (ms) |
|---|---|---|
| Host-to-device copies | 2 | 0.985312 |
| Vector-addition kernel | 1 | 0.177664 |
| Device-to-host copy | 1 | 2.057824 |
| Sum of listed operations | 4 | 3.220800 |

3.043136 ms in total, about 17.1 times the kernel duration. Their share of the listed GPU operation time was

$$\frac{0.985312 + 2.057824}{3.220800} \times 100\% = 94.5\%. \tag{2}$$

This percentage uses the summed copy and kernel durations, not the application's wall-clock time. Even if the kernel took no time at all, the combined duration would improve by only $3.220800/3.043136 = 1.058$ times. Reusing device allocations and keeping intermediate data on the device would therefore help more than optimizing this addition kernel.

The CPU-side API table showed a different picture. Three `cudaMalloc` calls totaled 76.152224 ms, or 92.4% of the recorded CUDA API time, and the slowest allocation alone took 75.425376 ms. This is consistent with a large first-use initialization cost, which belongs to this first execution and should be reported separately from steady-state work. The three `cudaMemcpy` API calls totaled 4.513568 ms, but this is CPU-side call time and not the GPU copy duration. API and GPU durations should not be added together, because their intervals can overlap.

In the profiled run, the program printed an event interval of 0.835424 ms, while Nsight Systems recorded only 0.177664 ms of kernel execution. CUDA events measure the elapsed time between the points where the GPU reaches them [3], so the interval can include GPU idle time while the host prepares and submits a first launch. The 0.657760 ms difference is close to the 0.653760 ms CPU duration of `cudaLaunchKernel`, which supports this explanation, although the summary tables alone do not show the exact overlap. We therefore do not treat the event interval as pure kernel time, and future comparisons should use the same warm-up and timing procedure.

## 2.5 Memory Stride: Timing and Hardware Counters

The stride experiment used the same $N$, block size, and grid size as vector addition. Each thread computed

$$B[i] = A[is] + 1, \qquad s \in \{1, 32\}. \tag{3}$$

The input allocation was large enough for the maximum stride and was initialized to zero. Output writes stayed contiguous in both cases. Each stride had an untimed warm-up launch and a synchronization before its timed launch.

```
nvcc -arch=sm_87 --ptxas-options=-v memory_stride.cu -o memory_stride 2>&1 | tee
    memory_stride_compile.txt
./memory_stride | tee memory_stride_output.txt
```

The compiler reported 10 registers per thread, no spills, and 376 bytes of constant memory. The standalone measurements were 0.134176 ms for stride 1 and 0.589664 ms for stride 32, so

$$\frac{t_{32}}{t_1} = \frac{0.589664}{0.134176} \approx 4.39. \tag{4}$$

Both spot checks returned `b[100] = 1.0`. Each time is a single post-warm-up sample, not an average, and we give no uncertainty estimate.

We then ran Nsight Compute with the resolved executable path, because `sudo ncu` did not find the program in its search path:

```
sudo "$(command -v ncu)" --set basic ./memory_stride 2>&1 | tee memory_stride_ncu.txt
```

The four profiled launches were the stride-1 warm-up, the stride-1 timed launch, the stride-32 warm-up, and the stride-32 timed launch. Table 3 compares the second and fourth. Stride 32 was slower even though its

Table 3: Nsight Compute results for the two timed launches.

| Metric | Stride 1 | Stride 32 |
|---|---|---|
| Profiler kernel duration ($\mu$s) | 149.95 | 586.14 |
| Theoretical occupancy (%) | 100 | 100 |
| Achieved occupancy (%) | 81.07 | 89.92 |
| Memory throughput (% of peak) | 52.25 | 82.19 |
| L2 cache throughput (% of peak) | 52.25 | 82.19 |
| Compute throughput (% of peak) | 17.08 | 3.48 |
| Registers per thread reported by profiler | 16 | 16 |

achieved occupancy was higher. Its memory-throughput percentage was also higher, which shows that greater use of the memory system does not mean more useful data per byte moved. The reported memory throughput equaled the L2 value in both cases, so it should not be read as DRAM bandwidth. The basic output warned that an optional DRAM metric was unavailable.

The profiler reported 16 registers per thread, while the compiler output reported 10. We keep both values with their sources, because the available logs do not explain the difference. Both strides had the same profiler-reported launch resources and 100% theoretical occupancy, so this measurement gives no support for a register-resource difference between them.

The basic collection replayed each launch in eight passes [5]. During profiling, the application's event intervals grew to hundreds of milliseconds, so we excluded those intervals and their printed ratio from the standalone speed comparison.

To get direct evidence of coalescing, we collected global-load requests and sectors separately:

```
sudo "$(command -v ncu)" --metrics l1tex__t_requests_pipe_lsu_mem_global_op_ld.sum,
    l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum ./memory_stride 2>&1 | tee
    memory_stride_sectors.txt
```

Each warp requested 32 four-byte values, or 128 useful bytes. When the reads are aligned and contiguous, these values fit in four 32-byte sectors. At stride 32, neighboring threads access addresses 128 bytes apart, so

Table 4: L1/TEX global-load counters; warm-up and timed launches agreed.

| Counter | Stride 1 | Stride 32 |
|---|---|---|
| Load requests | 32,768 | 32,768 |
| Load sectors | 131,072 | 1,048,576 |
| Sectors per request | 4 | 32 |

the same useful values need 32 sectors. The useful-byte fractions at the request level were

$$\eta_1 = \frac{128}{4 \times 32} = 100\%, \qquad \eta_{32} = \frac{128}{32 \times 32} = 12.5\%. \tag{5}$$

The counters directly show eight times as many load sectors per request. They do not prove eight times as many DRAM bytes, because cache behavior and the lower memory levels need separate measurements. Contiguous output stores and other costs also remain in both kernels, so an eightfold sector increase need not cause an eightfold slowdown. The 4.39-fold standalone slowdown is consistent with the loss of read coalescing. Both cases read the same number of useful elements, but stride 32 spans a larger input address range.

## 2.6 Runtime Monitoring with tegrastats

Initial 500 ms samples showed an idle GPU, temperatures near 51 °C, and device input power near 6.2 W. Because our kernels run for less than a millisecond, we repeated the monitoring at 100 ms intervals while launching the program repeatedly from a second terminal:

```
# Terminal A
tegrastats --interval 100 | tee tegrastats_active.txt
# Terminal B
for i in {1..100}; do ./memory_stride > /dev/null; done
```

The captured file had 315 samples, and 82 of them showed nonzero GPU activity. The highest `GR3D_FREQ` utilization was 55%. The sample at 16:22:29 on October 21 recorded 55% GPU utilization, a GPU temperature of 51.125 °C, RAM use of 3926/7607 MB, and a current `VDD_IN` power of 7523 mW.

This shows GPU activity during the repeated workload. It is not a measurement of occupancy or of the power of one isolated kernel. Each process also spent time on initialization, allocation, and cleanup, and the input initialization generated GPU work of its own. The sample therefore cannot tell stride 1 from stride 32, and the device input power includes more than the GPU.

## 2.7 Lab Questions and Connection to the Midterm

**Where did vector addition's time actually go?** It depends on what we measure. In the first-execution CUDA API summary, allocation dominated, and one long allocation is consistent with initialization overhead. Among the recorded GPU operations, data movement dominated: the copies took 3.043136 ms, or 94.5% of the combined duration, while the addition kernel took 0.177664 ms. We report first-use API costs and GPU operation costs separately for this reason.

**Why was stride 32 slower with the same kernel instructions?** The instruction structure and the number of useful elements did not change, but neighboring threads now read scattered addresses. Global-load requests stayed at 32,768 while sectors rose from 131,072 to 1,048,576, or from four to 32 sectors per request. This

lower read efficiency accompanied a 4.39-fold standalone slowdown, and the higher achieved occupancy did not make up for it.

**Connection to the midterm.** The midterm kernels can be analyzed with the same process, and we did so there. In transpose, sector counters showed that tiling cut L1/TEX store sectors from 8,388,608 to 2,097,152 and raised the useful bytes per store sector from 8 to 32, the same kind of coalescing evidence as the stride experiment. Bank-conflict counters were needed to judge padding, which reduced shared-load conflicts but increased shared-store conflicts. In tiled multiplication, L1/TEX load sectors fell about eightfold while L2 read sectors did not, so a reduction at one memory level did not carry to the next, and the $32 \times 32$ tile lost performance to lower occupancy. The common lesson is to profile the suspected mechanism before crediting a speedup to it, and to keep warm-up and timing boundaries consistent.

## 3 Overall Conclusion

The lab established a hardware budget, fixed the operating clocks, and combined application-level tracing with kernel-level counters. Vector addition showed that a short kernel can be a small part of the measured work: explicit copies accounted for 94.5% of the recorded GPU operation time, and first-use allocation dominated the CPU-side CUDA API time. The gap between the event timing and the traced kernel duration also showed why we must know exactly what a timing interval includes.

The stride experiment gave direct evidence that memory-access layout matters even when the useful work and launch configuration stay the same. Stride 32 needed eight times as many load sectors per request and ran 4.39 times slower in the standalone measurement. Its higher achieved occupancy and memory-system utilization did not mean better performance. Future optimization should target the measured bottleneck and confirm the explanation with the right counters. Repeated timing trials and full-array correctness checks would strengthen these single-sample, spot-checked results.

## References


[1] A. Doboli, "CUDA Profiling," ESE 366/ESE 566 laboratory slides, Stony Brook University, supplied course material.

[2] T. Li, "ESE366-CUDA-Profiling," source code, commit `a4d7bb367b202c1fa3f8f605ba8168c53a924af0`. Available: https://github.com/Timili0826/ESE366-CUDA-Profiling/tree/a4d7bb367b202c1fa3f8f605ba8168c53a924af0/Codes.

[3] NVIDIA, *CUDA C++ Best Practices Guide*, sections on timing, coalesced access, and occupancy. Available: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html. Accessed October 22, 2026.

[4] NVIDIA, *Nsight Systems User Guide*. Available: https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Accessed October 22, 2026.

[5] NVIDIA, *Nsight Compute Profiling Guide*, metric collection and replay. Available: https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html. Accessed October 22, 2026.